Dynamic Programming

> → **Key Idea**: Break problem into subproblems and store results to avoid recomputation

> • **When to use**: Optimization problems, overlapping subproblems, optimal substructure

> ⊕ **Complexity**: Time/Space vary by problem dimension (1D: O(n), 2D: $O(n^2)$)

> ⚠ **Common Pitfall**: Not identifying optimal substructure or inefficient state representation

Overview

Dynamic Programming (DP) solves problems by breaking them into smaller subproblems and storing results to avoid redundant calculations. It works when problems have optimal substructure and overlapping subproblems.

Key steps: 1) Define state, 2) Identify base cases, 3) Find recurrence relation, 4) Determine computation order.

Visual Example

DP State Transition (Fibonacci)

```
Recursive (overlapping):          DP (memoized):
       fib(5)                        fib(5)
      /      \                      /      \
   fib(4)   fib(3)              fib(4)   fib(3) ← cached!
   /    \    /    \             /    \
fib(3) fib(2) fib(2) fib(1)  fib(3) fib(2)
  ...    ...                     ↓
                               cached!

Time: O(2^n) → O(n)
```

2D DP Grid (Unique Paths)

```
Start → [1] [1] [1] [1]
        [1] [2] [3] [4]
        [1] [3] [6] [10] ← End

dp[i][j] = dp[i-1][j] + dp[i][j-1]
Number of ways to reach each cell from top-left
```

Longest Common Subsequence (LCS)

```
Text1: "ABCDE"    Text2: "ACE"

DP Table:
      ""  A  C  E
  ""   0  0  0  0
  A    0  1  1  1
  B    0  1  1  1
  C    0  1  2  2
  D    0  1  2  2
  E    0  1  2  3  ← LCS length = 3

Match: dp[i][j] = dp[i-1][j-1] + 1
No match: dp[i][j] = max(dp[i-1][j], dp[i][j-1])
```

0/1 Knapsack Visualization

```
Items: [(weight=2, value=3), (w=3, v=4), (w=4, v=5)]
Capacity W = 5

DP Table (max value for capacity):
Item\Cap   0   1   2   3   4   5
  none     0   0   0   0   0   0
  (2,3)    0   0   3   3   3   3
  (3,4)    0   0   3   4   4   7  ← take both (2,3)+(3,4)
  (4,5)    0   0   3   4   5   7

Answer: dp[3][5] = 7 (items 1 and 2)
```

Edit Distance State Transition

```
word1: "horse"  →  word2: "ros"

DP Table (min operations):
      ""  r  o  s
  ""   0  1  2  3
  h    1  1  2  3
  o    2  2  1  2
  r    3  2  2  2
  s    4  3  3  2
  e    5  4  4  3  ← Answer: 3 operations

Operations: Replace h→r, Delete r, Delete e
```

Core Implementation

1D DP - Climbing Stairs

Problem: You can climb 1 or 2 steps at a time. How many distinct ways to climb n stairs?

```
public int climbStairs(int n) {
    if (n <= 2) return n;

    int prev2 = 1, prev1 = 2;
    for (int i = 3; i <= n; i++) {
        int curr = prev1 + prev2;
        prev2 = prev1;
        prev1 = curr;
    }
    return prev1;
}
```

2D DP - Longest Common Subsequence (LCS)

Problem: Find the length of the longest subsequence common to two strings.

```
public int longestCommonSubsequence(String text1, String text2) {
    int m = text1.length(), n = text2.length();
    int[][] dp = new int[m + 1][n + 1];

    for (int i = 1; i <= m; i++) {
        for (int j = 1; j <= n; j++) {
            if (text1.charAt(i - 1) == text2.charAt(j - 1)) {
                dp[i][j] = dp[i - 1][j - 1] + 1;
            } else {
                dp[i][j] = Math.max(dp[i - 1][j], dp[i][j - 1]);
            }
        }
    }
    return dp[m][n];
}
```

Common Problem Patterns

1. 0/1 Knapsack

Problem: Given weights and values of n items, put items in a knapsack of capacity W to maximize total value.

```java
public int knapsack(int[] weights, int[] values, int W) {
    int n = weights.length;
    int[] dp = new int[W + 1]; // Space optimized

    for (int i = 0; i < n; i++) {
        // Traverse backwards to avoid using same item twice
        for (int w = W; w >= weights[i]; w--) {
            dp[w] = Math.max(dp[w], dp[w - weights[i]] + values[i]);
        }
    }
    return dp[W];
}
```

2. Longest Increasing Subsequence (LIS)

Problem: Find the length of the longest strictly increasing subsequence.

```java
// O(n log n) solution using binary search
public int lengthOfLIS(int[] nums) {
    List<Integer> tails = new ArrayList<>();

    for (int num : nums) {
        int pos = Collections.binarySearch(tails, num);
        if (pos < 0) pos = -(pos + 1);

        if (pos == tails.size()) {
            tails.add(num);
        } else {
            tails.set(pos, num);
        }
    }
    return tails.size();
}

// O(n²) classic DP solution
public int lengthOfLIS_DP(int[] nums) {
    int n = nums.length;
    int[] dp = new int[n];
    Arrays.fill(dp, 1);
    int maxLen = 1;

    for (int i = 1; i < n; i++) {
        for (int j = 0; j < i; j++) {
            if (nums[j] < nums[i]) {
                dp[i] = Math.max(dp[i], dp[j] + 1);
            }
        }
        maxLen = Math.max(maxLen, dp[i]);
    }
```

```
        return maxLen;
    }
```

3. Edit Distance (Levenshtein)

Problem: Find minimum operations (insert, delete, replace) to convert word1 to word2.

```
public int minDistance(String word1, String word2) {
    int m = word1.length(), n = word2.length();
    int[][] dp = new int[m + 1][n + 1];

    // Base cases
    for (int i = 0; i <= m; i++) dp[i][0] = i;
    for (int j = 0; j <= n; j++) dp[0][j] = j;

    for (int i = 1; i <= m; i++) {
        for (int j = 1; j <= n; j++) {
            if (word1.charAt(i - 1) == word2.charAt(j - 1)) {
                dp[i][j] = dp[i - 1][j - 1];
            } else {
                dp[i][j] = 1 + Math.min(dp[i - 1][j - 1],  // Replace
                               Math.min(dp[i - 1][j],      // Delete
                                        dp[i][j - 1]));    // Insert
            }
        }
    }
    return dp[m][n];
}
```

4. Coin Change

Problem: Find minimum number of coins to make up amount.

```
public int coinChange(int[] coins, int amount) {
    int[] dp = new int[amount + 1];
    Arrays.fill(dp, amount + 1);
    dp[0] = 0;

    for (int i = 1; i <= amount; i++) {
        for (int coin : coins) {
            if (coin <= i) {
                dp[i] = Math.min(dp[i], dp[i - coin] + 1);
            }
        }
    }
```

```
        return dp[amount] > amount ? -1 : dp[amount];
    }
```

5. House Robber

Problem: Rob houses along a street (can't rob adjacent houses). Maximize total amount.

```
public int rob(int[] nums) {
    if (nums.length == 1) return nums[0];

    int prev2 = 0, prev1 = 0;
    for (int num : nums) {
        int curr = Math.max(prev1, prev2 + num);
        prev2 = prev1;
        prev1 = curr;
    }
    return prev1;
}
```

Interview Tips

Pattern Recognition

- "Maximum/minimum" → likely DP
- "Count ways" → likely DP
- "Can you reach/achieve" → DP or greedy
- Substring/subsequence problems → often 2D DP

Common Mistakes

- Off-by-one errors in array indices
- Forgetting base cases
- Not considering empty input
- Using wrong iteration direction in space-optimized solutions

Space Optimization Template

```
// Convert 2D to 1D when dp[i] only depends on dp[i-1]
// Original: dp[i][j] = f(dp[i-1][j], dp[i-1][j-1])
// Optimized: iterate j backwards to preserve previous row values
for (int i = 1; i <= m; i++) {
    for (int j = n; j >= 1; j--) {  // Backwards!
```

```
            dp[j] = f(dp[j], dp[j-1]);
        }
    }
```

Time & Space Complexity

Problem Type	Time Complexity	Space Complexity
1D Linear DP	O(n)	O(n) → O(1) optimized
2D Grid DP	O(mn)	O(mn) → O(n) optimized
LCS/Edit Distance	O(mn)	O(mn) → O(min(m,n))
Knapsack	O(nW)	O(W)
LIS	$O(n^2)$ → O(n log n)	O(n)

Most DP problems can be space-optimized by observing that we often only need previous states.

← Previous: Greedy Algorithms | Back to Main Guide | Next: Graph Algorithms →